



Universidad
Indoamérica

UNIDAD 4 · FUNDAMENTOS DE SISTEMAS OPERATIVOS

Procesos y memoria

Universidad Tecnológica Indoamérica

Laboratorio TD3

El proceso: un programa en ejecución y su contexto

- Contexto: memoria, registros, estado de los descriptores...
- El SO ejecuta muchos procesos a la vez (unidad 1, A1.4)
- Cada uno tiene su **espacio de direcciones privado**: aislamiento, seguridad y la abstracción de memoria virtual
- Le oculta el núcleo y los demás procesos

Direccionamiento virtual

- Una dirección de n bits permite 2^n direcciones
- Espacio de direcciones = las direcciones + el estado de cada celda
- Cada proceso ve **solo el suyo**

En x86-64 hoy

Punteros de 64 bits, pero solo **48** se usan: 256 TiB virtuales, la mitad baja para el proceso y la alta para el núcleo. Con paginación de 5 niveles, 57 bits (128 PiB).

Las zonas de un proceso: pila, heap, datos, código

Salida real

```
$ ./zones
zone          address      perm  region
code   (main)   0x5f935f9d1350  r-xp  ~/fso/tds/bin/td3/zones
data   (initialized) 0x5f935f9d5010  rw-p  ~/fso/tds/bin/td3/zones
bss    (global_array) 0x5f935f9d5040  rw-p  ~/fso/tds/bin/td3/zones
heap   (malloc 10 B)  0x5f938dc96010  rw-p  [heap]
heap   (end: sbrk(0)) 0x5f938dcb6fff  rw-p  [heap]
mmap   (malloc 64 MiB) 0x71b340fff010  rw-p  (anonymous)
libc   (printf)       0x71b345064080  r-xp  /usr/lib/x86_64-linux-gnu/libc.so.6
stack  (local)        0x7fff6c1e68b4  rw-p  [stack]
```

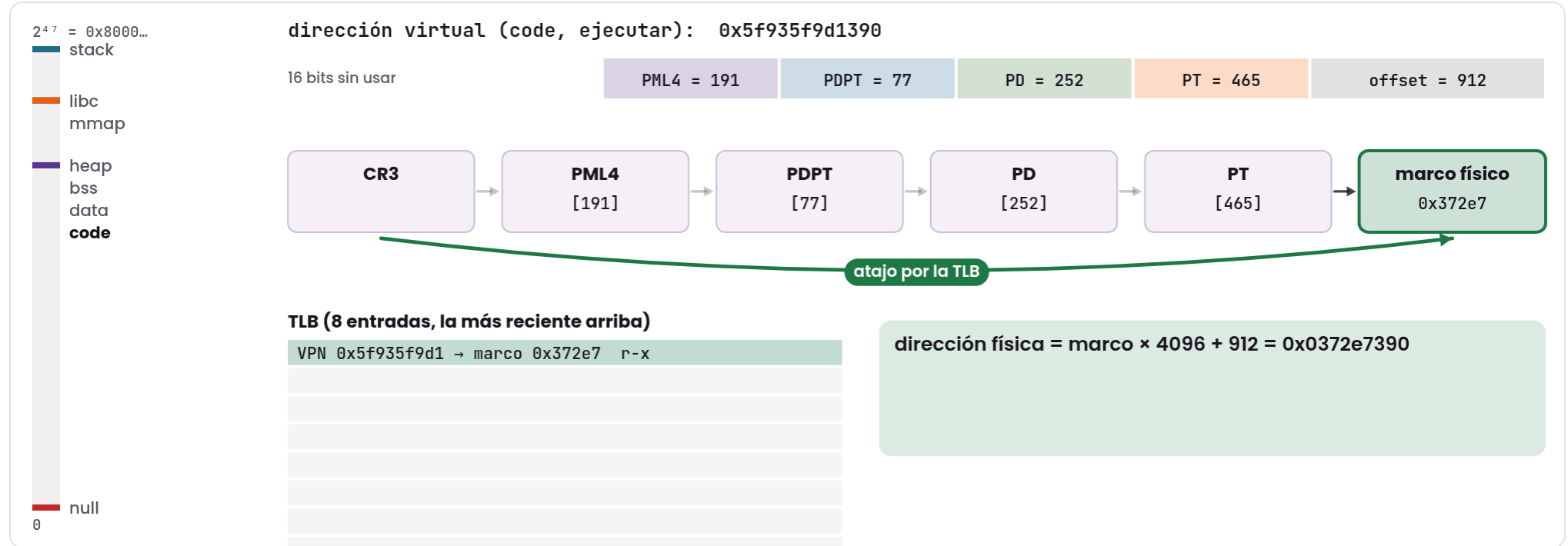
Pila: variables locales · **heap:** `malloc` pequeños · **mmap:** `malloc` grandes y bibliotecas · **datos/bss:** globales y `static` · **código** (texto): las instrucciones, de solo lectura y ejecutables.

Del virtual al físico: paginación

- La traducción la hace la CPU: la **MMU**, con la caché **TLB**
- La memoria se gestiona en **páginas**: 4 KiB en x86-64 (y páginas enormes de 2 MiB o 1 GiB); en ARM también 16 o 64 KiB
- Cada página virtual apunta a un **marco** físico, mediante una tabla (en x86-64, de 4 niveles)
- La dirección se parte en campos: índices de la tabla y desplazamiento

Corrección al original: decía "bloques de 2 KB (4 KB o 2 MB en x86)". En x86 la página base es de 4 KiB; 2 KB no es un tamaño de página de x86.

Traducir direcciones reales



Asignación perezosa: la página existe cuando se toca

Salida real

```
$ ./lazy 1024
start                               VmRSS =    1756 KiB   page faults so far = 0
malloc(1024 MiB) returned 0x7fadb67ff010
after malloc                         VmRSS =    1952 KiB   page faults so far = 8
after touching half of the pages    VmRSS =   526300 KiB  page faults so far = 131081
pages touched: 131072 (4096 bytes each)  page faults: 131073   1968 ns per fault
after free                           VmRSS =    2012 KiB   page faults so far = 131081
```

- Mucho más eficiente que asignar todo por adelantado
- ¿Y si no quedan marcos libres? Una página poco usada se lleva a disco (swap) o se comprime en RAM (zswap, zram): reemplazo de páginas →

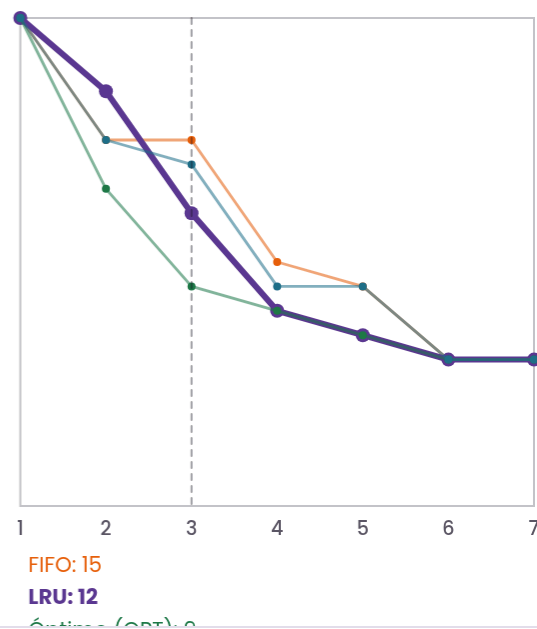
Sin marcos libres: ¿qué página sale?

LRU · 3 marcos

ref.	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
marco 0	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
marco 1		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
marco 2			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
	F	F	F	F		F		F	F	F	F			F		F		F		
				-7		-1		-2	-3	-0	-4			-0		-3		-2		

F = fallo · -x = víctima

fallos según los marcos



La interfaz de memoria: `brk`, `malloc`, `free`

C

```
int brk(void *end);           /* end of the heap */
void *sbrk(intptr_t delta);
void *malloc(size_t size);    /* NULL on failure */
void *calloc(size_t n, size_t size);
void *realloc(void *p, size_t size);
void free(void *p);           /* free(NULL): no-op */
```

No uses `brk` / `sbrk` directamente: usa `malloc`.

- glibc: bloques pequeños en el heap (`brk`); desde 128 KiB, `mmap` propio (se ve en `zones`)
- Linux es optimista (*overcommit*): `malloc` casi nunca devuelve NULL; si la memoria se agota, el **OOM killer** termina procesos
- Doble `free` o `free` de algo no reservado: comportamiento indefinido
- Depurar: GDB, Valgrind y hoy `-fsanitize=address` (ASan)

El proceso para el SO: el PCB

Estático

PID, PPID, usuario dueño, derechos de acceso

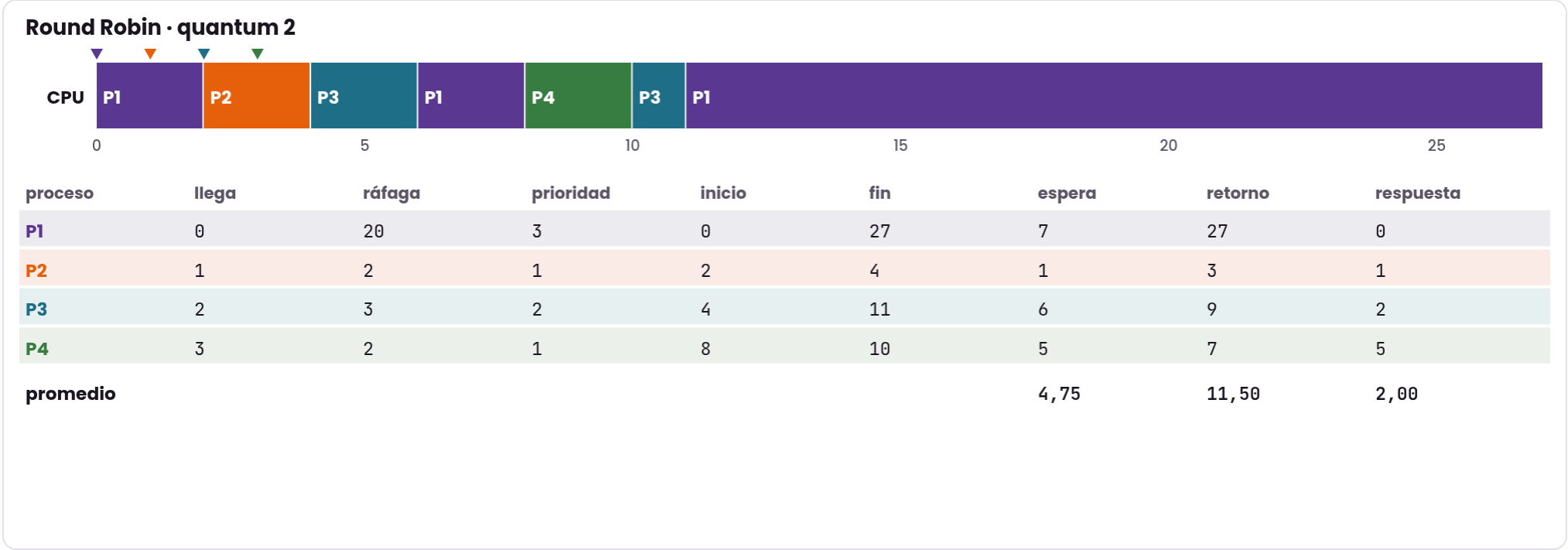
Dinámico

Estado, registros y contador de programa, prioridad, tiempos de CPU (usuario/sistema), descriptores abiertos, límites (`getrlimit`), señales, tabla de páginas, hilos

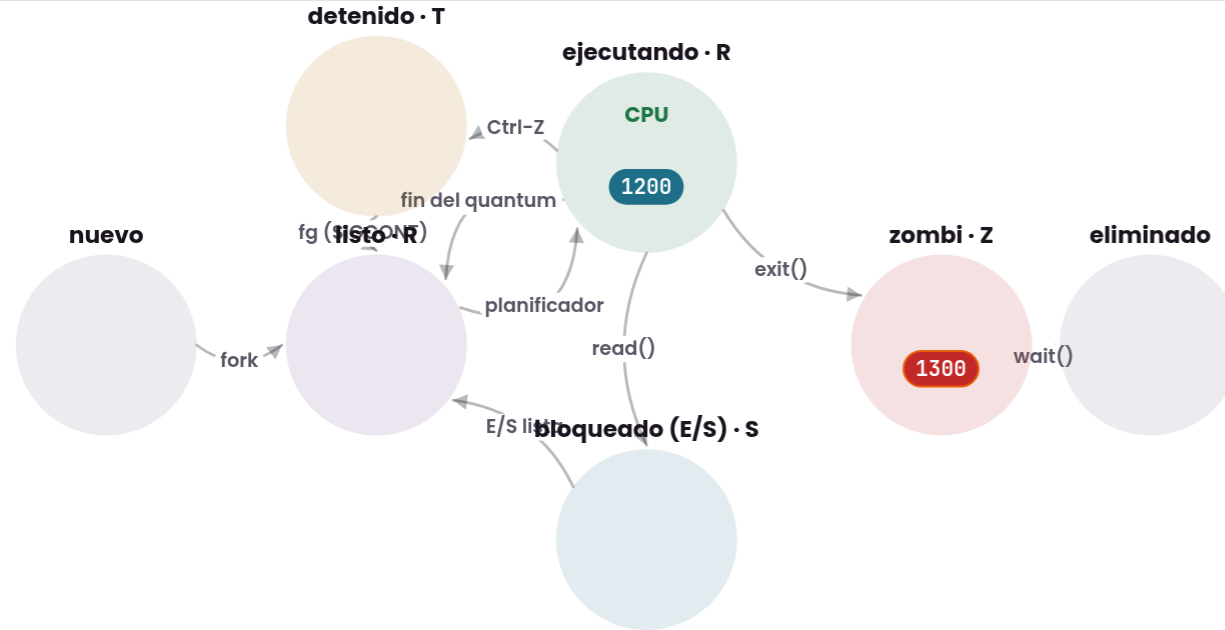
En Linux

El PCB es `struct task_struct` (varios KiB). Se inspecciona en `/proc/PID/`: `status`, `maps`, `fd/`, `limits` ... Así trabajan `ps` y `top`.

Cambio de contexto y planificación



Estados de un proceso



```
$ ps -o pid,ppid,stat,comm
  PID   PPID  STAT COMMAND
  1200   1100  R+   bash
  1300   1200  Z    lifecycle <defunct>
```

eventos

- bash (1200) hace fork → nace 1300
- la E/S de 1200 terminó (interrupción) → listo
- 1300 termina con estado 7: queda zombi hasta que su padre haga wait

¿Cómo se crea un proceso?

Windows

`CreateProcess()` crea el proceso y carga el programa en un solo paso, con muchas opciones.

Unix

1. `fork()` : un clon del proceso actual
2. El clon ajusta lo que quiera (descriptores, directorio...)
3. `exec()` : se transforma en otro programa

Cuatro familias: **crear** (`fork`), **ejecutar** (`exec...`), **terminar y señales** (`exit` , `kill` → U6), **esperar** (`wait...`).

Hoy también: `posix_spawn()` (fork + exec en una llamada), `vfork()` y `clone()` / `clone3()` , la llamada real detrás de `fork` y de los hilos (A4.7).

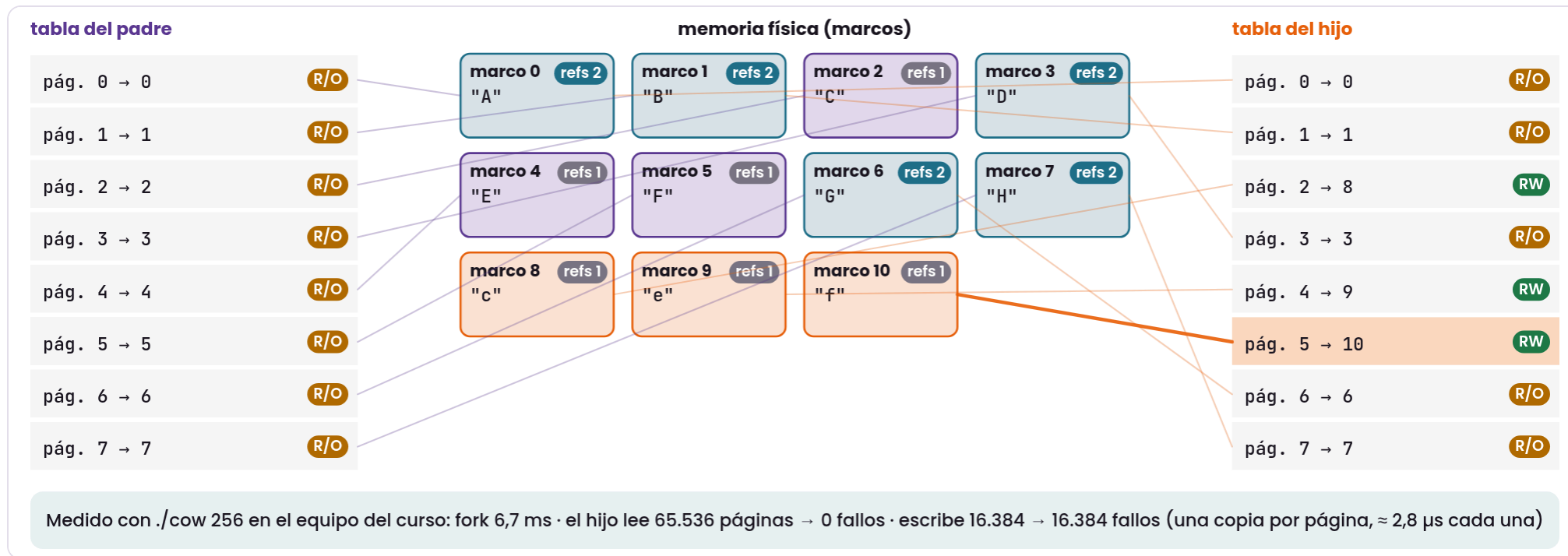
Clonar un proceso: `fork` y su costo

- El hijo ejecuta el mismo programa con una **copia** del espacio de direcciones del padre
- Crear el PCB es barato; copiar todas las páginas sería carísimo
- **Copia en escritura**: se comparten y se copian solo al escribir

Salida real

```
$ ./cow 256
parent: 256 MiB filled (65536 pages of 4096
bytes)
fork took 6.70 ms (only the page tables are
copied)
child reads 65536 pages: 0 page faults
(shared, nothing copied; sum 7864320)
child writes 16384 pages: 16384 page faults
(one copy per page), 2753 ns per copy
parent: the child exited with status 0; my
copy still says 'x'
```

Copia en escritura, página por página



pid_t fork(void)

- Devuelve **0 en el hijo**, el **PID del hijo en el padre** y **-1** si falla
- Idénticos salvo PID y PPID (`getpid`, `getppid`)
- El hijo hereda manejadores y máscara de señales, **no** las señales pendientes
- También hereda... el búfer de stdio pendiente

C tds/td3/fork_pid.c

```
pid_t pid = fork();
exit_if(pid == -1, "fork");

if (pid == 0) {                                /* child */
    printf("child %d, parent %d\n", getpid(), getppid());
    exit(EXIT_SUCCESS);                        /* exit flushes the child's buffer */
}

/* Parent: prints nothing after fork, but must wait for its child. */
int status;
exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
```

Salida real

```
$ ./fork_pid_0 | cat
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

Búferes de stdio y fork

```
printf("My pid is %d / ", getpid());
pid = fork();
/* child */ printf("child %d, parent %d\n", ...); exit(0);
/* parent */ waitpid(pid, ...); return 0;
```

el padre sale con exit(): vacía su búfer

padre · stdout: completo

terminó

(vacío)

hijo · stdout: completo

terminó

(vacío)

stderr: sin búfer

llamadas al sistema

```
[hijo] write(1, "My pid is 356446 / child 356448, parent 356446\n", 47)
write(1, "My pid is 356446 / ", 19)
```

tubería → cat → terminal

```
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

salida real (run.sh):

```
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

Ejecutar otro programa: `execve`

C

```
int execve(const char *path, char *const argv[], char *const envp[]);  
/* execl, execlp, execl, execv, execvp, execvpe:  
   l = list, v = vector, p = search PATH, e = environment */
```

- `argv` : los argumentos de `main` ; `envp` : el entorno (`PATH` , `HOME` , `LANG` , `LD_LIBRARY_PATH` ...; `getenv` / `setenv`)

- Si funciona, **no vuelve**: reemplaza código, datos, heap y pila
- Conserva PID, PPID y los descriptores abiertos (salvo los `O_CLOEXEC`)
- Con el bit SUID, el UID efectivo pasa a ser el del dueño del archivo (`passwd` , `sudo`)
- Si falla, devuelve -1

fork + dup2 + exec + wait : así lanza el shell

launch · PID 356524

ejecutando

ls -l demo · PID 356526

recogido por wait

descriptores del hijo

```
0 → terminal
1 → out.txt
2 → terminal
3 → -
```

strace -f (traza real, resumida)

```
356524 execve("./launch", ["/launch", "out.txt", "ls", "-l", "demo"], ...) = 0
356524 openat(AT_FDCWD, "out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
356524 write(3, "ls", ...) ...
356524 clone(..., flags=...|SIGCHLD) = 356526
356524 close(3) = 0
356524 wait4(356526, <unfinished ...>
356526 dup2(3, 1) = 1
356526 close(3) = 0
356526 execve("/usr/bin/ls", ...) = 0
356526 ... 19 llamadas de ls (openat, getdents64, close...) ...
356526 write(1, "a.txt\nb.txt\n", 12) = 12
356526 exit_group(0) = ?
356524 <... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) = 0}], 0, NULL) = 356526
356524 --- SIGCHLD {si_signo=SIGCHLD, ...} ---
356524 write(2, "launch: ls exited with status 0\n", ...)
```

palabra de estado de wait

0x0000

```
WIFEXITED(s) = true
WEXITSTATUS(s) = 0
WIFSIGNALED(s) = false
```

Si todo proceso nace de un `fork`, ¿quién es el primero?

Proceso 0

Uno por CPU (*idle*), construido por el núcleo, en modo núcleo: inicializa la memoria virtual, crea los hilos del núcleo (`kthreadd`, PID 2) y el proceso 1.

Proceso 1

Pasa a modo usuario y ejecuta `/sbin/init`, que hoy es **systemd**: arranca los servicios y adopta a los huérfanos.

Salida real

```
$ uname -srm; grep PRETTY_NAME /etc/os-release; ps -p 1 -o comm=; lsmod | tail -n +2 | wc -l; nproc; systemd-analyze
Linux 7.0.0-34-generic x86_64
PRETTY_NAME="Ubuntu 26.04 LTS"
systemd
...
```

Terminar un proceso: `_exit` y `exit`

C

```
void _exit(int status);    /* system call */  
void exit(int status);    /* atexit + flush stdio + _exit */
```

El estado (0–255) lo recoge el padre con `wait`; el shell lo muestra en `$?`. Distinto de 0 = error. Ninguna de las dos vuelve.

- Se cierran los archivos y se liberan las páginas (salvo las compartidas)
- Los hijos vivos pasan a `init` o al *subreaper* más cercano
- El padre recibe `SIGCHLD` (ignorada por omisión)
- Si era líder de sesión con terminal, los demás reciben `SIGHUP`
- Prefiere `exit`; usa `_exit` en el hijo tras un `exec` fallido

Sesiones y servicios (demonios)

- Una **sesión** agrupa procesos (típicamente, los de una terminal) con un líder
- `setsid()`: crea una sesión nueva sin terminal de control; -1 si ya es líder
- Dentro de la sesión, **grupos de procesos** (un trabajo del shell: `a | b`): `setpgid` / `getpgid`; `getsid` y `tcgetsid` consultan la sesión de un proceso y de una terminal
- Demonio clásico: `fork`, el padre sale, el hijo llama a `setsid`

Actualización 2026

En Linux los servicios los lanza **systemd**: el programa corre en primer plano y systemd se encarga de la sesión, los registros (`journalctl`), los reinicios y los límites (cgroups). Ya no hace falta "demonizarse".

Esperar a un hijo: `wait` y `waitpid`

C

```
pid_t wait(int *status);           /* = waitpid(-1, status, 0) */
pid_t waitpid(pid_t pid, int *status, int options);
/* pid > 0: that child · -1: any child · 0 / < -1: a process group
   options: WNOHANG (do not block) · WUNTRACED (stopped by SIGSTOP,
   SIGTSTP, SIGTTIN, SIGTTOU) · WCONTINUED (resumed by SIGCONT) */
```

Un hijo terminado queda **zombi** hasta el `wait`:

Salida real

```
$ ./lifecycle
1) the child 356667 has exited, but nobody has
called wait yet:
      PID    PPID STAT COMMAND
      356667 356666 Z    lifecycle
waitpid → WIFEXITED = 1, WEXITSTATUS = 7;
now ps finds nothing:
      PID    PPID STAT COMMAND
...
```

¿Cómo terminó? Las macros W...

WIFEXITED → WEXITSTATUS	terminó con <code>exit</code>
WIFSIGNALED → WTERMSIG	lo mató una señal
WIFSTOPPED → WSTOPSIG	detenido (con <code>WUNTRACED</code>)
WIFCONTINUED	reanudado (con <code>WCONTINUED</code>)

C tds/td3/launch.c

```
int status;
exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
if (WIFEXITED(status)) {
    fprintf(stderr, "launch: %s exited with status %d\n",
            return WEXITSTATUS(status);
}
if (WIFSIGNALED(status)) {
    fprintf(stderr, "launch: %s killed by signal %d (%s",
            signame(WTERMSIG(status)), WCOREDUMP(status)
    return 128 + WTERMSIG(status);
}
/* sh
```

TD3 resuelto + desafío con IA

- Las 5 variantes de `buffer` en terminal y en tubería
- `fork_pid` y sus tres soluciones; `launch` con tres finales
- Nuevo: `zones`, `lazy`, `cow` y `lifecycle` (mediciones)

Bash

```
make -C tds && tds/tests/run.sh td3
```



Universidad
Indoamérica

PARA LLEVAR

Tres preguntas

1. ¿Por qué `fork` de un proceso de 256 MiB tarda milisegundos?
2. ¿Qué conserva un proceso después de `execve`?
3. ¿Qué es un zombi y quién lo elimina?